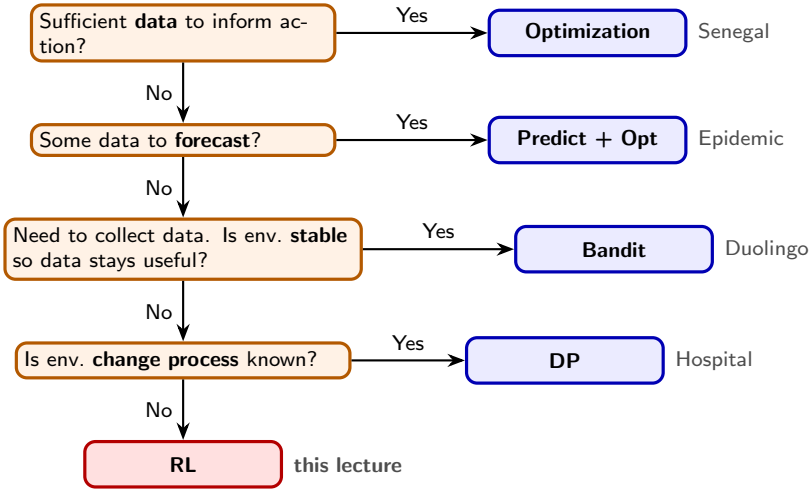
Lecture 5: Reinforcement Learning and Q-Learning

Peter Zhang

12768/19867/94867: From Data to Action

Recap of Course So Far: How to Go From Data to Actions



1. Learning by Doing
2. From Bandits to Reinforcement Learning
3. Q-Learning
4. A Worked Example: Driving on a Line
5. Backward Induction vs Q-Learning
6. So What: Lessons for Running AI Products
7. Optional: Beyond Tabular Q-Learning

Digression...

- Lots of language data recorded: internet, books, ...
  $\rightarrow$ Modern ML can use them to train language models to predict next tokens and paragraphs

- How about activities that require more tacit knowledge that is *not* recorded?
  - Autonomous driving, advanced manufacturing, ...
  - we see that these technologies are indeed taking longer to perfect
  - ...because without sufficient data, we need "**learning by doing**"

Learning by Doing

- Aristotle (350 BC): "For the things we have to learn before we can do them, we learn by doing them"
- Xunzi (300 BC): "I hear and I forget. I see and I remember. I do and I understand."
- Wright (1936): found that every time total aircraft production doubled, the required labor time for a new aircraft fell by 20%
- Arrow (1962): *The Economic Implications of Learning by Doing*
- Solow (1997): *Learning from "Learning by Doing"*

Today's Manufacturing: 40,000 parts in a Single Car!

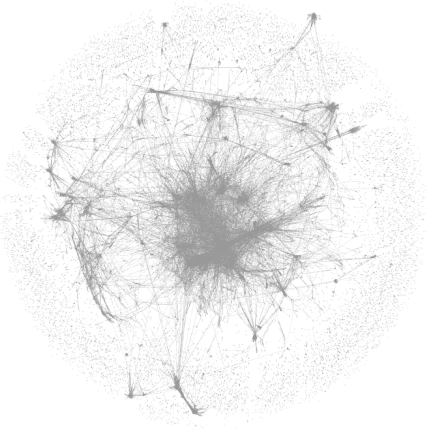
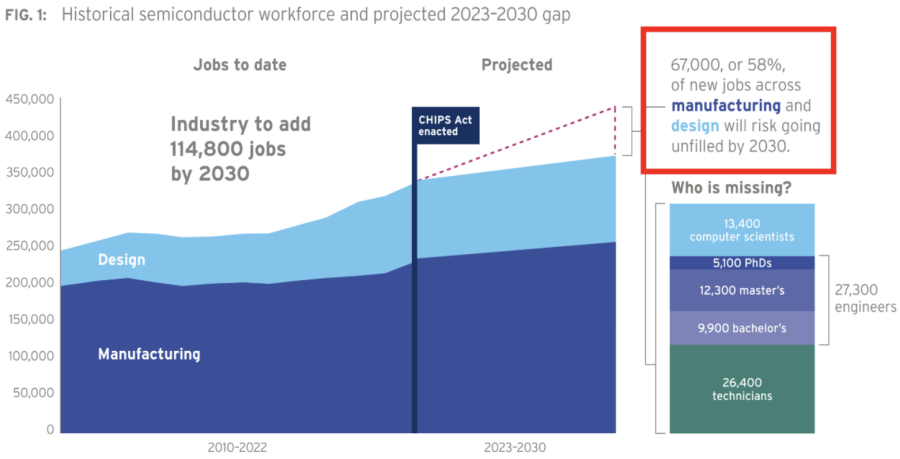


Figure 2-1: Overview of the bill-of-materials network with 74680 nodes and 97479 edges. Each node is a part or component in the manufacturing process, and edges represent the assembly requirements.

Today's Manufacturing: $>$ 1 Million Parts in a Plane!



Challenge in Manufacturing + Workforce Shortage



How to Fill the Gap?

- Workforce training
- Automation and robots
- ...
- *they all require some type of learning by doing* to get either the humans or the robots to work efficiently and work together

Today we formalize "learning by doing" in one particular way – via the so-called "reinforcement learning" approach.

Why You Cannot Just Write the Model Down

- A car has 40,000 parts. A plane has over a million.
- Last week, backward induction needed two things written down *before* we started: the transition $f_t(x, u)$ and the cost $g_t(x, u)$.
- There is effectively an infinite number of ways we can put the components together. Nobody can write them down completely.

1 Learning by Doing
2 From Bandits to Reinforcement Learning
3 Q-Learning
4 A Worked Example: Driving on a Line
5 Backward Induction vs Q-Learning
6 So What: Lessons for Running AI Products
7 Optional: Beyond Tabular Q-Learning

Bandit Problems: Our First Encounter of Reinforcement Learning

		Actions				
		a_1	a_2	a_3	...	a_n
State	s_1	0.15	0.0	**0.12**	...	0.0

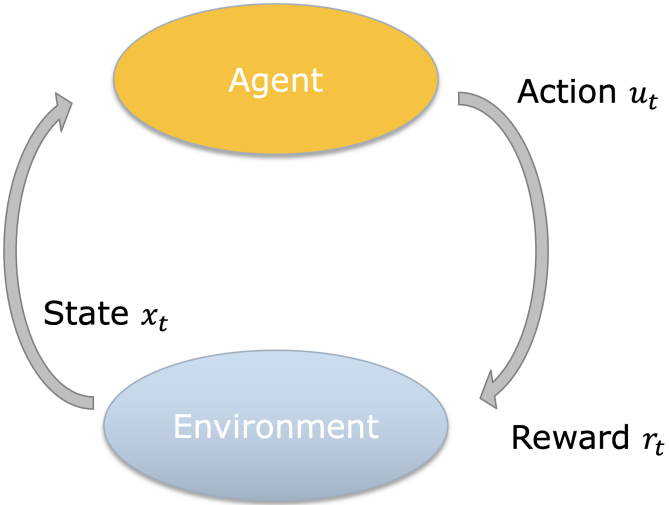
- **Trial 1, Exploration:** I'm pulling a_1 (arm 1) and observed reward $\mathbf{r} = \mathbf{0.15}$.
- **Trial 2, Exploration:** I try a_3 (arm 3) and observed reward $\mathbf{r} = \mathbf{0.12}$.
- **Trial 3, Exploitation:** ...
- ...

Bandits vs General RL

		Actions				
		a_1	a_2	a_3	...	a_n
State	s_1	0.15	0.0	0.12	...	0.0

- Bandits: balancing two goals
  1. Exploration: **completing** this entire action value table, by trying different actions
  2. Exploitation: **using** this action value table to guide actions
- General RL: same idea!
  1. Exploration: **completing** a larger action value table, by trying different actions
  2. Exploitation: **using** this action value table to guide actions

RL: Action and State Feedback Loop



RL: A Larger and Dynamic Table

We call this the "**quality table**", or Q table.

	Actions				
States	a_1	a_2	a_3	...	a_n
s_1	?	?	?	...	?
s_2	?	?	?	...	?
s_3	?	?	?	...	?
...					
s_n	?	?	?	...	?

1. Exploration: **completing** a larger action value table, by trying different actions
2. Exploitation: **using** this action value table to guide actions

RL: A Larger and Dynamic Table

States	Actions				
	a_1	a_2	a_3	...	a_n
s_1	0.15	0.1	0.12	...	0.1
s_2	0.05	0.20	0.08	...	0.28
s_3	0.11	0.37	0.25	...	0.34
...					
s_n	0.09	0.22	0.13	...	0.16

- Each entry in the table represents the expected cumulative reward of taking action a while in state s
- A different state $\rightarrow$ a different set of values
- Taking an action $\rightarrow$ state changes

Sample Path

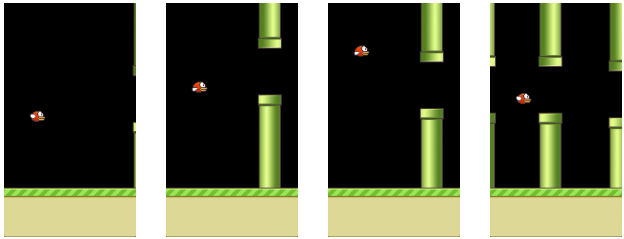
	Actions				
	a_1	a_2	a_3	...	a_n
s_1	0.15	0.1	0.121	...	0.1
s_2	0.05	0.20	0.08	...	0.28
s_3	0.11	0.355	0.25	...	0.34
...					
s_n	0.09	0.22	0.13	...	0.16

- Current state $s_1 \rightarrow$ take action $a_3 \rightarrow$ transit to state s_3
- Current state $s_3 \rightarrow$ take action $a_2 \rightarrow$ transit to state s_2
- Current state $s_2 \rightarrow$ take action ...

This is an example of "Q learning".

1 Learning by Doing
2 From Bandits to Reinforcement Learning
3 Q-Learning
4 A Worked Example: Driving on a Line
5 Backward Induction vs Q-Learning
6 So What: Lessons for Running AI Products
7 Optional: Beyond Tabular Q-Learning

Learning to Play, With Nobody Explaining the Game



2,000 tries | 20,000 | 200,000 | 2,000,000

- Nobody told this agent that gravity exists, or that pipes are solid, or that the floor kills you. It was told only whether it was still alive.
- Nothing changed between these four pictures except **how many times it played**.

So what is the rule that runs two million times? That is the next slide.

One thing to name now: this agent sees raw pixels, so its table would be far too big to store. A neural network holds the values instead. The update rule is the one we are about to write down.

RL: Q-Learning

1: Initialize $Q(s, a)$ for all states s and actions a
2: **for** each episode **do**
3: Initialize state s
4: **while** not terminal **do**
5: **Choose action** a (ϵ-greedy; UCB; softmax, ...)
6: Take this action a, observe reward r and next state s'
7: **Update Q-table:**

$$\text{New value} \leftarrow (1 - \alpha) \cdot \text{Old Value} + \alpha \cdot \text{New Observation:}$$

$$Q(s, a) \leftarrow (1 - \alpha) \cdot Q(s, a) + \alpha \cdot \left(r + \gamma \max_{a'} Q(s', a') \right)$$

8: Set $s \leftarrow s'$
9: **end while**
10: **end for**

Two Equivalent Ways to Represent The Update Rule

Blend form. Old estimate and new observation, mixed by α:

$$Q(s,a) \leftarrow (1-\alpha)\, Q(s,a) + \alpha\Big[\, r + \gamma \max_{a'} Q(s',a') \,\Big]$$

Correction form. Take a step of size α towards the target:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\, \underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{target}} - \underbrace{Q(s,a)}_{\text{current}} \,\Big]$$

These are **the same equation**. Expand the second one and the $Q(s,a)$ terms collect into $(1-\alpha)Q(s,a)$.

The bracket in the second form is the **temporal difference error**: how wrong the current estimate was. If it is zero, nothing moves. You will see exactly that happen in a few slides.

Costs or Rewards? min or max?

Cost convention

r is a *cost*, we want it small

$$\cdots + \alpha \Big[r + \gamma \min_{a'} Q(s', a') \Big]$$

act by min

Reward convention

r is a *reward*, we want it large

$$\cdots + \alpha \Big[r + \gamma \max_{a'} Q(s', a') \Big]$$

act by max

Three Knobs: γ, α, ϵ

γ, the discount factor. How much a future cost counts relative to an immediate one.

- $\gamma = 1$: a dollar at 8pm counts exactly as much as a dollar now. Fine for a problem that ends, like a 12 hour day. The hospital notebook uses $\gamma = 1$.
- $\gamma < 1$: later matters less. The driving example uses $\gamma = 0.9$.

α, the learning rate. How far we move towards the target.

- Large α tracks a changing world but never settles. Small α settles but learns slowly.
- So we **decay** it: move fast early, then stop overwriting good estimates with noise. The notebook decays linearly to a floor.

ϵ, the exploration rate. With probability ϵ, ignore the table and try something.

Q-Learning: Practical Design Choices

1. **How to initialize s?**
   Fixed initial state if the problem always starts the same (e.g., hospital shift starts at 8am and waiting room is always empty to start with). Randomized starts if you need to plan for different starting states (e.g., varying market conditions at 9:30am for stock markets).
2. **How to set the terminal state?**
   Use a fixed time horizon when planning resources (e.g., end of the day for staffing). Use event-based / episodic termination when there is a typical "end" event (e.g., campaign ends once budget is exhausted).
3. **How to select the exploration vs. exploitation strategy?**
   ϵ-greedy is simple for quick pilots. Decaying ϵ fits learning systems that need stability (e.g., repeated nonprofit outreach). UCB or softmax useful where fairness and balance are important (e.g., clinical trial recruitment).
4. **How to choose the learning rate α?**
   Larger α for fast-changing environments (e.g., online advertising with shifting trends). Smaller α for stable, slow-to-change systems (e.g., call center operations).

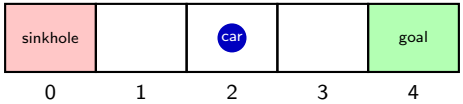
One More Choice, and Recitation Picks It Up

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\Big[r + \gamma \underbrace{\max_{a'} Q(s',a')}_{\text{this is a choice}}\Big]$$

- The max says: *assume I will behave greedily next period.*
- But with ϵ-greedy I will **not** always behave greedily. Some of the time I will do something random.
- So the update is valuing a policy that is not the one I am actually following.
- Replace $\max_{a'} Q(s',a')$ with $Q(s',a')$ for the action a' I *actually take next*, and you get a different algorithm called **SARSA**.

1 Learning by Doing
2 From Bandits to Reinforcement Learning
3 Q-Learning
4 A Worked Example: Driving on a Line
5 Backward Induction vs Q-Learning
6 So What: Lessons for Running AI Products
7 Optional: Beyond Tabular Q-Learning

The Track



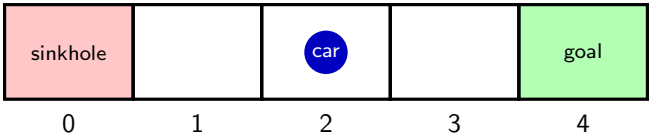
- **States:** positions 0, 1, 2, 3, 4.
- **Actions:** Left or Right, at positions 1, 2, 3 only. Positions 0 and 4 end the episode.
- **Rewards:** -1 for each move, -10 for falling in the sinkhole at 0, $+10$ for reaching the goal at 4.
- The car spawns at position 2.

- $\alpha = 0.1, \quad \gamma = 0.9$
- All Q-values start at 0.
- Reward convention, so we use max.

Position	Left (L)	Right (R)
1	0	0
2	0	0
3	0	0

We drive Right for a while, then try some other moves. Watch *one cell* change each step.

Let Us Fill This In Together



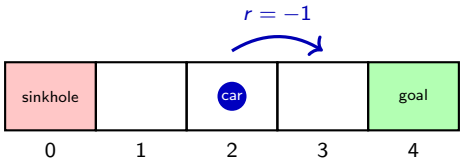
Position	Left	Right
1		
2		
3		

$$Q(s,a) \leftarrow (1-\alpha)\, Q(s,a) + \alpha\big(r + \gamma \max_{a'} Q(s',a')\big)$$

- $\alpha = 0.1, \quad \gamma = 0.9,$ everything starts at 0
- -1 per move, -10 in the sinkhole, $+10$ at the goal

The next slides have every step worked out, so nobody has to copy anything down.

Step 1: at 2, go Right, land on 3

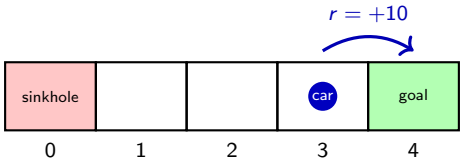


Position	Left (L)	Right (R)
1	0	0
2	0	**-0.1**
3	0	0

$$Q(2,R) \leftarrow (1-\alpha)Q(2,R) + \alpha\big(r + \gamma \max_{a'} Q(3,a')\big)$$
$$= 0 + 0.1 \times (-1 + 0.9 \times 0)$$
$$= -\mathbf{0.1}$$

Everything ahead is still 0, so all this step learns is "moving costs 1".

Step 2: at 3, go Right, reach the goal

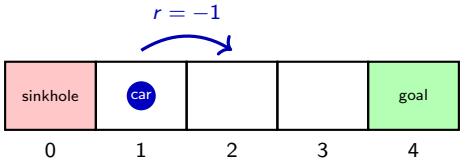


Position	Left (L)	Right (R)
1	0	0
2	0	-0.1
3	0	**1**

$$
\begin{aligned}
Q(3,R) &\leftarrow (1-\alpha)Q(3,R) + \alpha\big(r + \gamma \max_{a'} Q(4,a')\big) \\
&= 0 + 0.1 \times (10 + 0.9 \times 0) \\
&= \mathbf{1}
\end{aligned}
$$

The goal is worth $+10$, but we only move a tenth of the way there. That is α.

Step 3: respawn at 1, go Right, land on 2

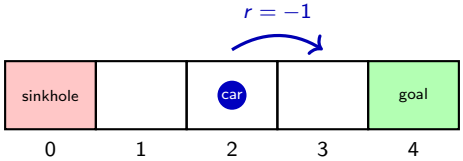


Position	Left (L)	Right (R)
1	0	**-0.1**
2	0	-0.1
3	0	1

$$Q(1,R) \leftarrow (1-\alpha)Q(1,R) + \alpha\big(r + \gamma \max_{a'} Q(2,a')\big)$$
$$= 0 + 0.1 \times (-1 + 0.9 \times 0)$$
$$= -\mathbf{0.1}$$

$\max_{a'} Q(2,a') = \max(0, -0.1) = 0$. The good news at position 3 has not reached back here yet.

Step 4: at 2, go Right again

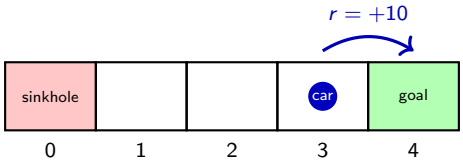


Position	Left (L)	Right (R)
1	0	-0.1
2	0	**-0.10**
3	0	1

$$Q(2,R) \leftarrow (1-\alpha)Q(2,R) + \alpha\big(r + \gamma \max_{a'} Q(3,a')\big)$$
$$= 0.9 \times (-0.1) + 0.1 \times (-1 + 0.9 \times 1)$$
$$= -0.09 + (-0.01) \;=\; \mathbf{-0.10}$$

First time a learned value feeds backward, through the max. And yet the cell does not move: the target is $-1 + 0.9(1) = -0.1$, exactly what $Q(2,R)$ already was. **The temporal difference error is zero**, so $\alpha \times 0 = 0$.

Step 5: at 3, go Right, reach the goal again

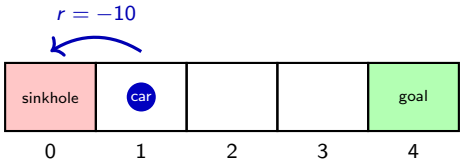


Position	Left (L)	Right (R)
1	0	-0.1
2	0	-0.10
3	0	**1.9**

$$Q(3,R) \leftarrow (1-\alpha)Q(3,R) + \alpha\big(r + \gamma \max_{a'} Q(4,a')\big)$$
$$= 0.9 \times 1 + 0.1 \times (10 + 0.9 \times 0)$$
$$= 0.9 + 1 \;=\; \mathbf{1.9}$$

Second visit to the same cell, so it climbs again: $0 \to 1 \to 1.9$. It is walking towards 10.

Step 6: respawn at 1, go Left, fall in

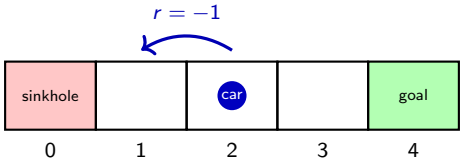


Position	Left (L)	Right (R)
1	**-1**	-0.1
2	0	-0.10
3	0	1.9

$$
\begin{aligned}
Q(1, L) &\leftarrow (1-\alpha)Q(1, L) + \alpha\big(r + \gamma \max_{a'} Q(0, a')\big) \\
&= 0 + 0.1 \times (-10 + 0.9 \times 0) \\
&= \mathbf{-1}
\end{aligned}
$$

The first bad news. The sinkhole finally gets recorded. Note we had to *fall in* to learn this. Nobody told the agent that 0 was dangerous; it found out the expensive way.

Step 7: respawn at 2, go Left, land on 1

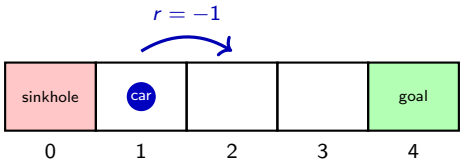


Position	Left (L)	Right (R)
1	-1	-0.1
2	**-0.109**	-0.10
3	0	1.9

$$Q(2, L) \leftarrow (1-\alpha)Q(2, L) + \alpha\big(r + \gamma \max_{a'} Q(1, a')\big)$$
$$= 0 + 0.1 \times (-1 + 0.9 \times (-0.1))$$
$$= 0.1 \times (-1.09) \;=\; \mathbf{-0.109}$$

$\max_{a'} Q(1, a') = \max(-1, -0.1) = -0.1$. The agent already prefers Right at position 1, so the -1 is *not* what propagates back. The max protects it.

Step 8: at 1, go Right, land on 2



Position	Left (L)	Right (R)
1	-1	**-0.199**
2	-0.109	-0.10
3	0	1.9

$$Q(1,R) \leftarrow (1-\alpha)Q(1,R) + \alpha\big(r + \gamma \max_{a'} Q(2,a')\big)$$
$$= 0.9 \times (-0.1) + 0.1 \times (-1 + 0.9 \times (-0.10))$$
$$= -0.09 + (-0.109) \;=\; \mathbf{-0.199}$$

$\max_{a'} Q(2,a') = \max(-0.109, -0.10) = -0.10$. Position 2 currently looks slightly bad, so position 1 gets worse too. Early estimates are pessimistic everywhere; the $+10$ has only propagated one step back so far.

Where It Settles

After 8 steps

Position	Left (L)	Right (R)
1	-1	-0.199
2	-0.109	-0.10
3	0	1.9

After many thousands of episodes

Position	Left (L)	Right (R)
1	-10	**6.2**
2	4.58	**8.0**
3	6.2	**10.0**

You can check the converged column by hand, working backwards from the goal:

$$Q^\star(3, R) = 10, \qquad Q^\star(2, R) = -1 + 0.9(10) = 8, \qquad Q^\star(1, R) = -1 + 0.9(8) = 6.2$$

That backward calculation is **exactly backward induction**. Q-learning reached the same answer without ever being told the rules of the track, just by driving on it many times.

LIVE DEMO

The five-cell track, step by step

`ql-bench.html` · tab 2, the five-cell track

Walk through the eight moves. One cell changes each time. Then run it to convergence and compare the table with the exact answer.

Two things worth trying afterwards: set γ to 0.5 and see how far the values drop, then raise ϵ and compare what happens to Q-learning and to SARSA.

Try It Yourself: What Changes the Final Q-Values?

1. Change the discount rate γ. The converged $Q^\star(\cdot, R)$ column:

	$\gamma = 0.5$	$\gamma = 0.9$	$\gamma = 1$
$Q^\star(3, R)$	10	10	10
$Q^\star(2, R)$	4	8	9
$Q^\star(1, R)$	1	6.2	8

A small γ makes the distant goal nearly worthless from far away. The *policy* here still says Right everywhere, but the values change a lot.

2. Change the learning rate α. It changes how fast you get there and how much the values wobble. It does **not** change where they settle.

3. Change the policy used to drive, for example ϵ-greedy instead of fixed or random. It changes *which cells get visited*, so it changes how fast each one converges, and whether rarely-tried cells converge at all.

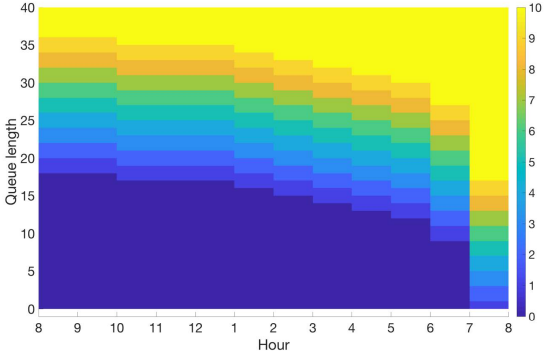
1 Learning by Doing
2 From Bandits to Reinforcement Learning
3 Q-Learning
4 A Worked Example: Driving on a Line
5 Backward Induction vs Q-Learning
6 So What: Lessons for Running AI Products
7 Optional: Beyond Tabular Q-Learning

Reload: The Hospital Surge Problem from Last Week

- A 12 hour day, hour by hour.
- **State:** patients waiting.
- **Action:** how many surge doctors to add this hour.
- **Costs:** \$30 per waiting patient per hour, \$250 per surge doctor, \$200 per patient still waiting at 8pm.
- 10 base doctors, each clearing 2 patients per hour.

The shape of the answer, which we computed by hand:

- Below about 8 waiting, buy nothing.
- Above that, roughly one extra doctor per 2 patients over the line.



We are not re-deriving it. We are asking: could we have found this *without* knowing the arrival distribution or the costs?

What Each Method Needs Before It Can Start

Backward induction

Needs, written down in advance:

- the arrival distribution
- the service rate
- every cost

Gives you the **exact** optimal policy, for every state and every hour, in one pass.

Q-learning

Needs only:

- the ability to act
- the ability to observe what happened

Gives you an **estimate**, which gets better with experience. And it needs a lot of experience.

Side by Side

	Backward induction	**Q-learning**
Model required	Yes. Transition probabilities and costs, all of them.	No. Learns from experience alone.
What you get	The exact cost-to-go $J_t(x)$ and the exact optimal policy.	An iterative estimate of $Q_t(x, u)$, approaching the optimum.
How it works	One sweep backward from the terminal condition.	Many samples forward, each nudging one cell.
Data cost	None. It is arithmetic.	High. Every cell needs to be visited many times.
Reach for it when	You can write the dynamics down and the state space is manageable.	You cannot write the dynamics down, but you can interact or simulate.

Backward induction is called **model-based**. Q-learning is **model-free**.

LIVE DEMO

The hospital, both methods side by side

`ql-bench.html` · tab 1, backward induction vs Q-learning

Two policy tables, the exact one and the learned one, and a third table showing how often each state was visited.

After 2,000 episodes the cost gap is about 46%. After 200,000 it is about 5%. The white cells in the third table are states the learner has never been in. Switch to the small problem to see the same training budget go much further.

1 Learning by Doing
2 From Bandits to Reinforcement Learning
3 Q-Learning
4 A Worked Example: Driving on a Line
5 Backward Induction vs Q-Learning
6 So What: Lessons for Running AI Products
7 Optional: Beyond Tabular Q-Learning

If you are a manager of analytical products...

Safe to delegate

- Why the update converges at all.
- The conditions on α that make it converge.
- Whether to use a replay buffer.
- PPO, GRPO, DQN, and which one your vendor picked.
- The network architecture.
- All of the code.

Engineers own these and can experiment on these

Not safe to delegate

- What the system had to *do* to learn.
- Who absorbs the cost of its mistakes while it learns.
- Whether those mistakes are recoverable.
- Whether you needed learning here at all.

Engineering team may not raise these questions...technical managers should ask.

1. Learning by Doing
2. From Bandits to Reinforcement Learning
3. Q-Learning
4. A Worked Example: Driving on a Line
5. Backward Induction vs Q-Learning
6. So What: Lessons for Running AI Products
7. Optional: Beyond Tabular Q-Learning

From Tabular Q-Learning to Deep Q-Learning

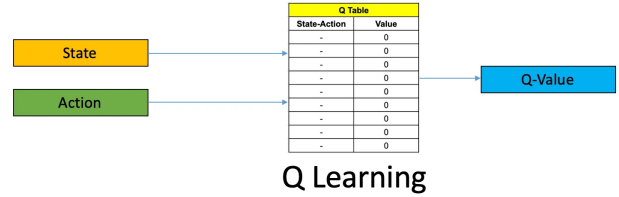


Image from https://becominghuman.ai/train-your-first-rl-agent-a-step-by-step-guide-353bced83722

Use Case: DoorDash Logistics Assignment

Which Dasher should fulfill which delivery?

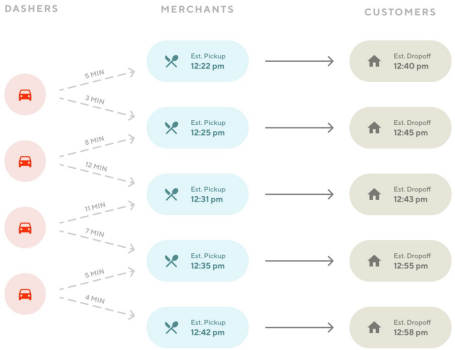


Image Credit: Richard Hwang et al.

Richard Hwang, Gary Ren, & Yixin Tang. (2018, September). Reinforcement Learning for On-Demand Logistics. Retrieved August 1, 2025, from https://careersatdoordash.com/blog/reinforcement-learning-for-on-demand-logistics/

Problem Setup

- **State**: Outstanding deliveries and working Dashers
- **Action**: Different variants of assignment algorithm with parameters
- **Reward**: Combination of delivery speeds, Dasher efficiency, and a penalty for undelivered deliveries
- **Agent**: A deep neural network for the Q function

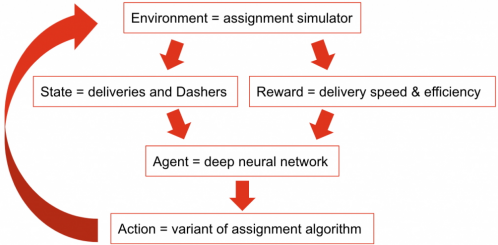


Image Credit: Richard Hwang et al.

Deep Q-Learning

- **Idea**: Approximate $Q(s, a)$ as state space is too big to compute
- **Model**: Multilayer perceptron (MLP)
- **Input**: High-dimensional featurized state
- **Output**: Predicted Q-value for each action

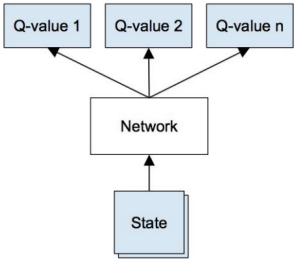


Image Credit: Richard Hwang et al.

Results & Takeaways

- The model picks the best assignment algorithm to use for each state over that same day and obtained the new average delivery speeds and Dasher efficiencies
- The model achieves on average a **6 second improvement in delivery speed** and **1.5 second improvement in Dasher efficiency** across all deliveries, compared to production algorithm in use
- Non-trivial for large operations: DoorDash completed 6 billion orders by early 2025, if 6 seconds saved per order $\rightarrow$ 1141 *years* saved in total

Other Methods

- Many other methods to tackle reinforcement learning
- Q $\rightarrow$ value function based learning
- Policy based
- Hybrid...